PYTHON – Module 2

Syntax • Variables • Data Types and Data Structures • Operators • Practical Exercises

Learning Objectives

Core concepts and practical application

- Read and write basic Python statements using correct syntax and indentation.
- Create variables and understand dynamic typing and naming rules.
- Recognise basic data types and choose suitable collection structures.
- Use operators for calculations, comparisons and logical conditions.
- Combine the concepts in short Python programs.

1. Python Syntax: The Rules

Syntax determines whether a Python statement is valid

- Python is case-sensitive: name and Name are different identifiers.
- Text values are enclosed in single or double quotes.
- print() displays values and messages.
- Statements are generally written one per line.
- Missing quotes, brackets or correct spelling can cause errors.

```
print("Welcome to Python")
name = "Arun"
print(name)
```

Output

```
Welcome to Python
Arun
```

Indentation Defines Code Blocks

Leading spaces show which statements belong together

```
score = 75

if score >= 50:
     print("Pass")
     print("Result recorded")

print("Program complete")
```

- The indented lines belong to the if block.
- Four spaces per indentation level is the standard convention.
- Statements in the same block need consistent indentation.
- Incorrect indentation can cause an IndentationError or change program logic.

Comments Explain Code

Comments are ignored when Python executes the program

```
# Calculate the total
price = 100
tax = 5
total = price + tax  # Add price and tax
print(total)
```

- A single-line comment begins with #.
- Comments document purpose, assumptions or important logic.
- Use comments to clarify code, not to repeat every obvious statement.

2. Variables: Names for Values

Assignment creates a variable; a separate declaration is not required

```
name = "Arun"
age = 25
salary = 35000.50

age = 26  # Reassignment
print(name, age, salary)
```

- A variable is a name that refers to a value.
- Python uses dynamic typing; a variable's type is determined by its current value.
- Variables can be reassigned to new values.
- Use meaningful names such as total_marks or student_name.

Variable Naming Rules

Readable names reduce mistakes

- Names may contain letters, digits and underscores.
- A name cannot begin with a digit.
- Names cannot be Python keywords such as if, for or class.
- Names are case-sensitive: score, Score and SCORE are different.
- Convention: use lowercase words separated by underscores.

```
# Valid
student_name = "Meera"
score2 = 88
_total = 10

# Invalid
2score = 88
student-name = "Meera"
class = "Python"
```

Multiple Assignment and Dynamic Typing

Python supports concise assignment

```
x, y = 10, 20
x = x + 5

value = 100
value = "one hundred"

print(x, y)
print(value)
```

- Multiple assignment assigns values to multiple variables in one statement.
- A variable can later refer to a value of another type.
- Dynamic typing does not mean values have no type; each value still has a type.

3. Basic Data Types

A data type describes the kind of value

Type	Purpose	Example
int	Whole numbers	count = 12
float	Numbers with a fractional part	price = 19.95
str	Text	city = "Pune"
bool	Logical value: True or False	is_valid = True
NoneType	Absence of a value	result = None

Inspecting and Converting Types

type() reports a value's type; conversion functions change compatible values

```
age = 25
price = 3.5
number_text = "250"

print(type(age))           # <class 'int'>
print(type(price))         # <class 'float'>
print(int(number_text) + 50)  # 300
print(float("2.5"))        # 2.5
print(str(25))             # "25"
```

- int() converts compatible values to integers.
- float() converts compatible values to floating-point numbers.
- str() converts a value to text.
- Invalid conversions raise an error; for example, int("hello") fails.

Collection Data Structures

Choose a collection based on how data needs to be stored and accessed

Structure	Key characteristics	Example	Mutable?
list	Ordered; allows duplicates	[10, 20, 20]	Yes
tuple	Ordered; allows duplicates	(10, 20, 20)	No
set	Unique elements; no positional indexing	{10, 20, 30}	Yes
dict	Key-value pairs; keys are unique	{"name": "Arun"}	Yes

List vs Tuple vs Set vs Dictionary

Comparison and quick memory guide

Characteristic	List	Tuple	Set	Dictionary
Purpose	Store a collection of items	Store items that should not be changed	Store unique items	Store key–value pairs
Syntax	[ ]	( )	{ }	{key: value}
Example	[10, 20, 30]	(10, 20, 30)	{10, 20, 30}	{"a": 10, "b": 20}
Ordered	Yes	Yes	No guaranteed order	Yes, preserves insertion order
Allows duplicates	Yes	Yes	No duplicate elements	Keys must be unique; values may repeat
Mutable?	Yes	No	Yes	Yes

QUICK MEMORY GUIDE

LIST
Sequence you can modify
`[ ]`

TUPLE
Sequence you should not modify
`( )`

SET
Unique items only
`{ }`

DICTIONARY
Look up values using keys
`{key: value}`

Exercise – Basic Data Types

```
# Basic data types
name = "Priya"
age = 21
percentage = 85.5
is_student = True

# Display basic data types
print("Name:", name)
print("Age:", age)
print("Percentage:", percentage)
print("Is Student:", is_student)
```

List, Tuple and Dictionary — Program Examples

LIST

```
student_marks = [78, 85, 92]

print("Marks:", student_marks)
print("First Mark:", student_marks[0])
print("Number of Marks:", len(student_mark
print("Data Type:", type(student_marks))
```

TUPLE

```
student_details = ("Priya", 21, 85.5)

print("Details:", student_details)
print("Name:", student_details[0])
print("Age:", student_details[1])
print("Percentage:", student_details[2])
print("Data Type:", type(student_details))
print("Number of Items:", len(student_details)
```

DICTIONARY

```
student_info = {
     "name": "Priya",
     "age": 21,
     "percentage": 85.5
}

print("Information:", student_info)
print("Name:", student_info["name"])
print("Age:", student_info["age"])
print("Percentage:", student_info["percentage"])
print("Data Type:", type(student_info))
```

Strings: Properties & Operations

1. Indexing

Access one character by position. Indexing starts at 0.

```
text = "PYTHON"
print(text[0])  # P
```

2. Immutability

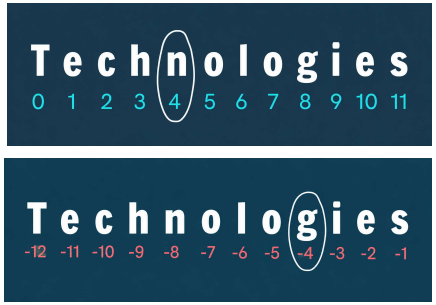
Strings cannot be changed in place; create a new string.

```
word = "cat"
# word[0] = "b"  # TypeError
word = "b" + word[1:]
```

3. Concatenation

Join strings using the + operator.

```
first = "Hello"
print(first + " Python")
# Hello Python
```



4. Escape sequences

Special sequences represent newline or tab characters.

```
print("Hello\\nPython")
print("Name:\\tAsha")
```

5. Slicing & negative index

Use [start:stop]; stop is excluded. Negative indexes count from the end.

```
text = "PYTHON"
print(text[1:4])  # YTH
print(text[-1])   # N
```

Remember: positions start at 0; negative indexing starts at −1; slicing excludes the stop position.

Sample 5 Python String Methods

STRING VARIABLE

```
text = "  Hello Python  "
```

1. upper()

Converts all letters to uppercase.

```
text.upper()
```

Output: ' HELLO PYTHON '

2. lower()

Converts all letters to lowercase.

```
text.lower()
```

Output: ' hello python '

3. strip()

Removes whitespace from both ends.

```
text.strip()
```

Output: 'Hello Python'

4. replace()

Replaces one substring with another.

```
text.replace("Python", "World")
```

Output: ' Hello World '

5. split()

Splits the string into a list of parts.

```
text.split()
```

Output: ['Hello', 'Python']

String methods return a result; they do not change the original string.

4. Operators in Python

Operators act on values and variables

Category	Operators	Purpose / example
Arithmetic	+ - * / // % **	10 // 3 → 3; 10 % 3 → 1; 2 ** 3 → 8
Comparison	== != > < >= <=	10 >= 5 → True
Assignment	= += -= *= /=	x += 2 updates x by adding 2
Logical	and or not	age >= 18 and age <= 60
Membership	in not in	"Py" in "Python" → True
Identity	is is not	value is None
Bitwise	& | ^ ~ << >>	Operate on integer bits

Arithmetic and Assignment Operators

Perform calculations and update values

```
a = 10
b = 3

print(a + b)   # 13
print(a / b)   # 3.333...
print(a // b)  # 3
print(a % b)   # 1
print(a ** b)  # 1000

a += 2
print(a)       # 12
```

- +, -, * perform addition, subtraction and multiplication.
- / performs true division; // performs floor division.
- % returns the remainder; ** raises a value to a power.
- Compound assignment such as += combines an operation with reassignment.

Comparison and Logical Operators

Conditions evaluate to True or False

```
age = 25
has_id = True

print(age >= 18)                # True
print(age < 18)                 # False
print(age >= 18 and has_id)     # True
print(age < 18 or has_id)       # True
print(not has_id)               # False
```

- Comparison operators compare values.
- and is True when both conditions are True.
- or is True when at least one condition is True.
- not reverses a Boolean result.

Membership Operators

Use membership operators to check whether a value exists in a sequence or collection.

1. in

Returns True if the value is present; otherwise, returns False.

2. not in

Returns True if the value is absent; otherwise, returns False.

3. Example

```
fruits = ["apple", "banana", "mango"]

print("banana" in fruits)
print("grape" in fruits)
print("grape" not in fruits)
```

Output

```
True
False
True
```

Remember

in = present | not in = absent

Membership checks also work with strings, tuples, sets and dictionaries. For dictionaries, membership checks keys.

Identity Operators and id()

Read in order: understand the concept → learn the operators → see the example

1. Object identity Identity tells us whether two variables refer to the very same object—not merely objects with equal values.

2. id(object) This function returns an integer identifier for an object. The identifier stays the same during that object's lifetime.

3. The identity operators

is → True when both names refer to the same object
is not → True when they refer to different objects

4. Do not confuse identity with equality

== compares values. is compares object identity.
Two separate lists can be equal without being the same object.

5. Example — concepts in action

```
a = [1, 2]
b = a              # b refers to the same list as a
c = [1, 2]         # c is a separate list

print(a == c)       # True: same values
print(a is b)       # True: same object
print(a is c)       # False: different objects
print(id(a) == id(b))  # True
```

Output

```
True
True
False
True
```

Remember

Use == for values; use is / is not for object identity.

Bitwise Operators — Quick Summary

Bitwise operators work on the binary bits (0s and 1s) of integers.

& AND

1 only if both bits are 1

```
5 & 3  →  1
```

| OR

1 if either bit is 1

```
5 | 3  →  7
```

^ XOR

1 if bits differ

```
5 ^ 3  →  6
```

~ NOT

Inverts bits; Python rule: ~n = -(n + 1)

```
~5  →  -6
```

<< Left shift

Moves bits left; for $n \geq 0$, $\times 2^k$

```
5 << 1  →  10
```

>> Right shift

Moves bits right; for $n \geq 0$, floor-divide by 2^k

```
5 >> 1  →  2
```

Binary example `5 = 0101` `3 = 0011`

`& gives 0001 (1)` | `gives 0111 (7)` `^ gives 0110 (6)`

Where are they used?

- Flags and permissions (turn/check settings)
- Networking and protocol headers
- Embedded hardware and registers
- Image/color data; cryptographic algorithms and checksums

Important distinction

Bitwise operators act on integer bits. Logical operators (and, or, not) work with truth values. Bitwise operations are less common in everyday Python, but useful in low-level and binary-data tasks.

Practical example: flags

`READ = 1 (001)` `WRITE = 2 (010)` `EXECUTE = 4 (100)`

```
permissions = READ | WRITE  →  3 (011)
permissions & READ  →  True     |     permissions & EXECUTE  →  False
```

Bitwise Operators - Understanding Binary Numbers

Binary is a base-2 number system that uses only two digits: 0 and 1.

Each position has a power-of-2 place value. From right to left: 1, 2, 4, 8, 16, ...

1. Decimal → Binary

Repeatedly divide by 2; record each remainder.

Division	Quotient	Remainder
5 ÷ 2	2	**1**
2 ÷ 2	1	**0**
1 ÷ 2	0	**1**

Read remainders bottom to top: 101

Decimal 5 = Binary 101

2. Binary → Decimal

Use place values 4, 2, 1 for binary 101.

Place value 4	Place value 2	Place value 1
1	**0**	**1**
1 × 4 = 4	**0 × 2 = 0**	**1 × 1 = 1**

Add the contributions:

`4 + 0 + 1 = 5`

So binary 101 = decimal 5

Remember: decimal → binary uses division by 2; binary → decimal adds the place values where the bit is 1.

5. Hands-on Practice

Apply the concepts in short programs

- Basic output: display a welcome message, a name and three lines of text.
- Variables: store a student’s name, age and marks; display the values and update the age.
- Data types: create int, float, str, bool and None values; print each type.
- Type conversion: convert "250" to an integer, add 50 and display the result.
- Collections: create a list, tuple, set and dictionary and access selected values.

Hands-on Practice: Operators and Combined Task

Write, run and verify each program

- Operators: for two numbers, display sum, difference, product, quotient, floor quotient, remainder and power.
- Comparisons: determine which number is greater and combine conditions using and/or.
- Combined task: store marks for three subjects, calculate total and average, then check whether the average is at least 50.

```
marks1 = 70
marks2 = 65
marks3 = 80

total = marks1 + marks2 + marks3
average = total / 3

print("Total:", total)
print("Average:", average)
print("Average >= 50:", average >= 50)
```

Summary

Core ideas to retain

- Syntax rules, indentation and comments make Python code readable and valid.
- Assignment creates variables; Python determines types dynamically.
- Basic types include int, float, str, bool and NoneType.
- Lists, tuples, sets and dictionaries support different ways to organise data.
- Operators perform calculations, comparisons, logical tests, membership and identity checks.
- Practice improves accuracy in reading, writing and debugging simple programs.